

# 《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

## 实验名称：

XSS 脚本攻击

## 实验要求：

复现课本第十一章实验三，通过 img 和 script 两类方式实现跨站脚本攻击，撰写实验报告。有能力者，可以自己撰写更安全的过滤程序。

## 实验过程：

跨站脚本攻击(XSS)是当前 web 应用中最危险和最普遍的漏洞之一。XSS 通常用于发动 cookie 窃取、恶意软件传播(蠕虫攻击)、会话劫持、恶意重定向等。在这种攻击中，攻击者将恶意 JavaScript 代码注入到网站页面中，这样受害者的浏览器就会执行攻击者编写的恶意脚本。

### 1. Script 方式

#### 1.1>编写 Dreamweaver 文件

```
<!DOCTYPE html>
<head><meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{   confirm("Congratulations~"); }
</script> </head>
<body>
<h1 align=center>--Welcome To The Simple XSS Test--</h1>
<?php
ini_set("display_errors", 0);
$str=strtolower( $_GET["keyword"]);
$str2=str_replace("script","", $str);
$str3=str_replace("on","", $str2);
$str4=str_replace("src","", $str3);
echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>".<center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value='".$str4.'">
</form>
</center>';
?>
</body>
</html>
```

输入以上代码，编写 php 网页。然后我们进入 [http://127.0.0.1/xss\\_test.php](http://127.0.0.1/xss_test.php) 地址查看，可以看到如下界面，说明代码正常运行：



### 1.2>黑盒测试

[http://127.0.0.1/xss\\_test.php](http://127.0.0.1/xss_test.php) 页面里有一个 Submit 按钮和输入框，并且还有标题提示 XSS。于是输入简单的 XSS 脚本：`<script>alert('xss')</script>`来进行测试。点击 Submit 按钮以后，效果如下：

## --Welcome To The Simple XSS Test--

Hello `<script>alert('xss')</script>`.

结果发现 Hello 后面出现了我们输入的内容，并且输入框中的回显过滤了 script 关键字，这个时候考虑后台只是最简单的一次过滤。于是可以利用双写关键字绕过，构造脚本：`<scrscripript>alert('xss')</scrscripript>`测试。执行效果如下：

## --Welcome To The Simple XSS Test--

Hello `<scrscripript>alert('xss')</scrscripript>`.

发现虽然输入框中的回显确实是我们想要攻击的脚本，但是代码并没有执行。为了判断问题到底出在哪里，我们查看源码，可以发现第 5 行重写了 alert 函数，如果可以成功执行 alert 函数的话，页面将会跳出一个确认框显示 Congratulations~，这是我们 XSS 成功攻击的标志，而此时没有触发弹窗。接着往下查看 16 行的标签，这是我们唯一能输入的地方，发现其发生改变：`<input name=keyword value="<script>alert('xss')</script>">`

可见，虽然成功的插入了`<script></script>`标签组，但是并没有跳出 input 的标签，使得我们的脚本仅仅可以回显而不能利用。想到需要将前面的标签闭合，于是构造如下脚本：`"><scrscripript>alert('XSS')</scrscripript><!--`。

此外，php 服务器会将双引号等符号转义以预防用户构造特殊输入进行攻击，于是将 `magic_quotes_gpc = On` 改为 `off`。此时再行调试，得到了想要的结果 Congratulations~，测试完成：



### 1.3>白盒测试

我们从源码的角度来看一下页面的核心逻辑。

```
<?php
ini_set("display_errors", 0);
$str = strtolower( $_GET["keyword"]);
$str2=str_replace("script","", $str);
$str3=str_replace("on","", $str2);
$str4=str_replace("src","", $str3);
echo "<h2 align=center>Hello ".htmlspecialchars($str).</h2>".<center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value=" . $str4 . ">
</form>
</center>';
?>
```

发现跟黑盒测试的情况差不多,但也有没测到的地方。比如 Hello 后面显示的值是经过小写转换的。输入框中回显值的过滤方法是将 script、on、src 等关键字都替换成了空,过滤的内容并不是很多。故攻击脚本的构造方法也有多种。

## 2. img 方式

### 2.1>用 img 标签构建脚本

```
<img src=ops! onerror="alert('XSS')">
```

<img>标签是用来定义 HTML 中的图像,src 一般是图像的来源。而 onerror 事件会在文档或图像加载过程中发生错误时被触发。所以上面这个攻击脚本的逻辑是,当 img 加载一个错误的图像来源 ops!时,会触发 onerror 事件,从而执行 alert 函数。

所以我们修改一下原有的代码,添加了一个 img 标签,并设置其 src 属性为一个无效的 URL ops!,并使用 onerror 事件来执行 JavaScript 代码。为了绕过过滤和替换操作,用字符编码方式代替 js 代码嵌入到 onerror 属性中,并通过 eval 函数执行解码后的代码。

```
<!DOCTYPE html>
<head><meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{   confirm("Congratulations~");  }
```

```
</script> </head>
<body>
<h1 align=center>--Welcome To The Simple XSS Test--</h1>
<?php
ini_set("display_errors", 0);
$str=strtolower( $_GET["keyword"]);
$str2=str_replace("script","", $str);
$str3=str_replace("on","", $str2);
$str4=str_replace("src","", $str3);
echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>".<center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value='".htmlspecialchars($str4)."'>
</form>
</center>';
?>

</body>
</html>
```

## 2. 2>运行结果



可见成功出发了弹窗。

### 心得体会：

通过实验，具体地接触了解了两种跨站脚本攻击方式——用 script 和用 img 的方式进行攻击。需要注意的是，php 会过滤很多转义字符，这是实验中需要解决的一个问题。此外，了解了 script 通过替换无效内容来过滤关键字的原理，以及 payload 的构建。